Protocolo de Encriptação Temporizada por Custo — Formalização

---

Visão Geral

Um arquivo F é encriptado com AES-256-GCM usando uma chave K derivada via HMAC-SHA256 de uma semente curta S de n bits. Um hash $V =$ HMAC-SHA256$(S,\ n)$ é publicado como compromisso de verificação — qualquer um pode testar candidatos contra V sem precisar do arquivo. A chave é derivada encadeando: $K =$ HMAC-SHA256$(S,\ V)$. Um único protocolo (HMAC-SHA256) para tudo.

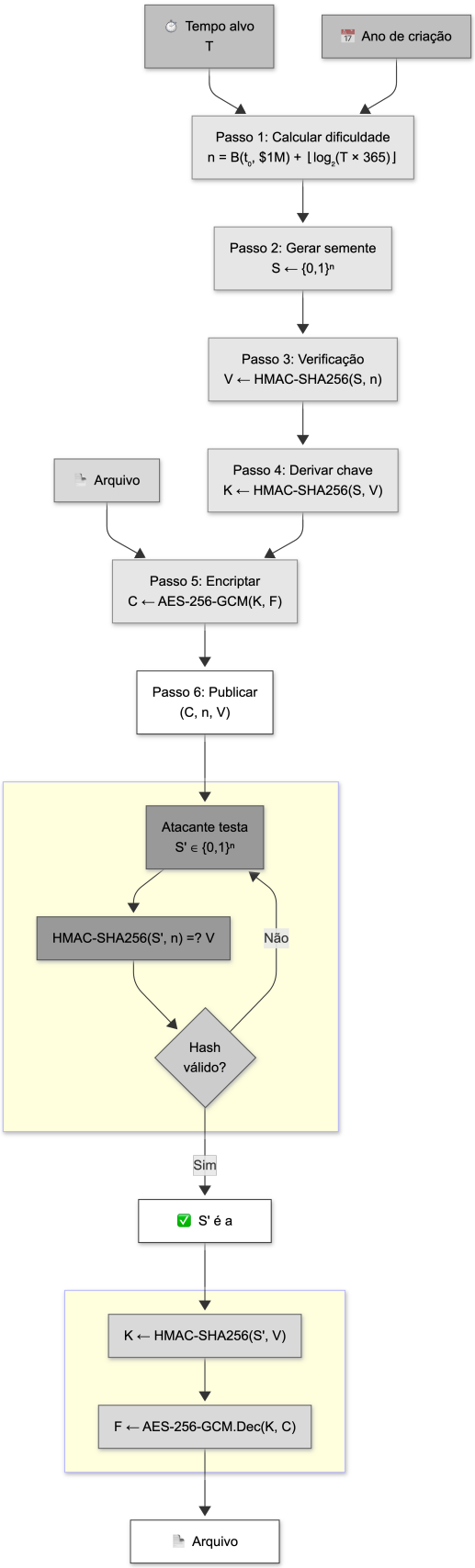

Figure 1: Diagrama do Protocolo Time Vault

Parâmetros de Entrada

Símbolo	Tipo	Descrição
F	$\{0,1\}^*$	File — arquivo original em bytes
T	$\mathbb{R}^+$	Time — tempo alvo em anos
t_0	$\mathbb{N}$	Time zero — ano de criação do puzzle

Construção do Puzzle

Passo 1 — Calcular a dificuldade:

$$n = \mathcal{B}(t_0,\ \$1\text{M}) + \lfloor \log_2(T) \rfloor$$

Parte	Significado
n	Tamanho da semente em bits — define a dificuldade do puzzle
$\mathcal{B}(t_0, \$1\text{M})$	Bits que $\$1$M em GPUs quebra em t_0 rodando por 1 ano
T	Tempo alvo em anos — fator de escala temporal
$\lfloor \log_2(T) \rfloor$	Converte o fator multiplicativo em bits adicionais (positivo ou negativo)

Intuição: $\mathcal{B}$ define quantos bits $\$1$M/ano alcança hoje. $T > 1$ ano soma bits (mais dificuldade); $T < 1$ ano subtrai bits (menos dificuldade). Duplicar T adiciona +1 bit.

Passo 2 — Gerar a semente:

$$S \xleftarrow{\$} \{0,1\}^n$$

Parte	Significado
S	Seed — semente aleatória de n bits; é o segredo do puzzle
$\xleftarrow{\$}$	Amostragem uniforme aleatória (o cifrão indica aleatoriedade criptográfica)
$\{0,1\}^n$	Conjunto de todas as strings binárias de comprimento n

Passo 3 — Hash de verificação:

$$V \leftarrow \text{HMAC-SHA256}(S,\ n)$$

Parte	Significado
V	Verification hash — compromisso público de S; permite verificar sem o arquivo
HMAC-SHA256	Código de autenticação baseado em hash — RFC 2104
S	Chave do HMAC (o segredo)
n	Mensagem do HMAC (parâmetro público)

Passo 4 — Derivar a chave:

$$K \leftarrow \text{HMAC-SHA256}(S,\ V)$$

Parte	Significado
K	Key — chave AES de 256 bits derivada do encadeamento $S \to V \to K$
S	Chave do HMAC (mesmo segredo)
V	Mensagem do HMAC (output do passo anterior)

Mesmo protocolo do Passo 3, com mensagem diferente. V alimenta K — encadeamento natural.

Passo 5 — Encriptar o arquivo:

$$C \leftarrow \text{AES-256-GCM}(K,\ F)$$

Parte	Significado
C	Ciphertext — arquivo encriptado + authentication tag GCM
AES	Advanced Encryption Standard — cifra de bloco simétrica
256	Tamanho da chave em bits
GCM	Galois/Counter Mode — encriptação autenticada (confidencialidade + integridade)

Passo 6 — Publicar:

$$(C,\ n,\ V)$$

Parte	Significado
C	Ciphertext (contém o tag GCM embutido)
n	Número de bits da semente — informa o espaço de busca
V	Hash de verificação — permite testar candidatos sem decriptar

Como o Atacante Quebra

Fase 1 — Busca (rápida, sem o arquivo):

$$\forall\, S' \in \{0,1\}^n : \quad \text{HMAC-SHA256}(S',\, n) \stackrel{?}{=} V$$

Cada teste é um único HMAC-SHA256.

Fase 2 — Decriptação (uma única vez, quando encontrar S'):

$$\begin{aligned} V &\leftarrow \text{HMAC-SHA256}(S',\, n) \\ K &\leftarrow \text{HMAC-SHA256}(S',\, V) \\ F &\leftarrow \text{AES-256-GCM.Dec}(K,\, C) \end{aligned}$$

Custo esperado: 2^n avaliações de HMAC-SHA256. Cada tentativa é independente — busca puramente paralela.

Fórmula de Custo

O custo para quebrar em 1 ano com o melhor hardware (GPUs) disponível:

$$\boxed{\text{Custo}(n,\, t_0) = \$1\text{M} \times 2^{(n-\mathcal{B}(t_0))}}$$

Como $n = \mathcal{B} + \lfloor \log_2(T) \rfloor$, isso simplifica para Custo $\approx \$1\text{M} \times T$.

Exemplos com $t_0 = 2026$, $\mathcal{B}(2026) = 67$:

Tempo alvo T	$\lfloor \log_2(T) \rfloor$	Bits n	Custo em 1 ano
1 semana	−5	62 bits	~$31.250
1 mês	−3	64 bits	~$125.000
6 meses	−1	66 bits	~$500.000
1 ano	0	67 bits	$1.000.000
10 anos	+3	70 bits	~$8.000.000
50 anos	+5	72 bits	~$32.000.000
100 anos	+6	73 bits	~$64.000.000

A escala é linear em dólares mas logarítmica em bits — duplicar o tempo alvo adiciona apenas 1 bit mas dobra o custo.

Propriedades do Protocolo

- Verificação sem o arquivo — V permite provar que S' é a solução sem precisar de C
- Um único protocolo — HMAC-SHA256 para verificação e derivação de chave
- Não há custódio — nenhum terceiro guarda S; ele existe apenas como espaço de busca
- Verificável publicamente — qualquer um com V pode confirmar que um S' é a solução
- Custo previsível — a função $\mathcal{B}$ permite estimar o custo de quebra em qualquer ano futuro
- Degradação natural — à medida que hardware evolui, o custo de quebra cai; isso é intencional e quantificável
- Busca puramente paralela — cada tentativa é independente; escala linearmente com hardware

---

Benchmark de Poder Computacional por Busca Exaustiva

Motivação

Comparar poder computacional ao longo do tempo exige uma tarefa de referência que seja:

1. Computacionalmente mensurável — custo teórico bem definido
2. Sem atalhos algorítmicos — não melhorável por criptoanalise
3. Paralelizável linearmente — dobrando hardware, dobra o progresso

A busca exaustiva sobre HMAC-SHA256 satisfaz essas propriedades: cada candidato exige uma avaliação de HMAC, sem atalho possível. O hardware relevante são GPUs — ASICs de Bitcoin (SHA-256d) não suportam HMAC-SHA256.

O Benchmark $\mathcal{B}(t, D)$

Seja:

- t — ano de referência
- D — orçamento em dólares
- $P(t, D)$ — número total de tentativas (HMAC-SHA256) executáveis com orçamento D no ano t, rodando por 1 ano contínuo
- $\mathcal{B}(t, D)$ — maior n tal que $2^n \leq P(t, D)$

$$\mathcal{B}(t, D) = \lfloor \log_2 P(t, D) \rfloor$$

Interpretação: $\mathcal{B}(t, D)$ é o número máximo de bits de semente que o orçamento D no ano t consegue buscar exaustivamente em 1 ano.

Aproximação Contínua de $\mathcal{B}$

A tabela histórica (1945–2025) é discreta. Para uso programático e extrapolação, aproximamos $\mathcal{B}(t, \$1\text{M})$ por uma função logística ajustada por mínimos quadrados:

$$\boxed{\mathcal{B}(t) = \frac{L}{1 + e^{-k(t - t_{\text{mid}})}} + b}$$

Parâmetro	Valor	Significado
L	$141{,}54$	Amplitude da curva (range em bits)
k	$0{,}0233$	Taxa de crescimento
t_{mid}	$1925{,}9$	Ponto de inflexão (metade do crescimento)
b	$-61{,}03$	Offset base

Propriedades da aproximação:

- MSE = 2,13 contra os 23 pontos da tabela histórica (RMSE $\approx 1{,}5$ bits)
- Assíntota superior: $L + b \approx 80$ bits — limite natural do hardware GPU acessível com $\$1\text{M}$
- Assíntota inferior: $b \approx -61$ bits — converge a zero para épocas pré-computacionais
- Ponto de inflexão: $t_{\text{mid}} = 1925{,}9$ — crescimento centrado no século XX
- Estagnação recente: a partir de ~2016, o custo/hash em GPUs estabilizou (~67 bits). GPUs ficam mais rápidas mas proporcionalmente mais caras

  A curva logística captura a forma de S dos dados: crescimento lento (1940s–70s), aceleração (1980s–2010s), e estagnação (2016+). ASICs de Bitcoin não se aplicam a HMAC-SHA256.

Definição de Equivalência Temporal

Dado $\mathcal{B}(t_1, D) = b_1$ e $\mathcal{B}(t_2, D) = b_2$, o custo em t_2 para replicar o poder de t_1 é:

$$C(t_1 \to t_2) = D \cdot 2^{b_1 - b_2}$$

Exemplo: $\mathcal{B}(1980, 1\text{M}) = 48$ e $\mathcal{B}(2025, 1\text{M}) = 67$

$$C(1980 \to 2025) = \$1\text{M} \cdot 2^{48-67} = \$1\text{M} \cdot 2^{-19} \approx \$1{,}91$$

Propriedades do Benchmark

Monotonicidade esperada: $\mathcal{B}(t+1, D) \geq \mathcal{B}(t, D)$ — o poder só cresce ou mantém.

Violações detectam anomalias reais: quedas em $\mathcal{B}$ (como 2000→2005 na tabela) indicam transições tecnológicas — de benchmarks genéricos para SHA-256 real — não ruído do benchmark.

Independência de domínio: o mesmo $\mathcal{B}$ serve para criptografia, IA, simulação — qualquer campo onde o custo computacional importa.

Aplicações

- Parametrização de esquemas criptográficos: definir n baseado no ano de implantação e vida útil esperada do sistema
- Protocolo de revelação temporizada: o prazo é expresso em bits de $\mathcal{B}$, não em tempo absoluto, tornando-o robusto a avanços de hardware

---

Integração com Bitcoin — Bounty On-Chain e Prova de Quebra

Bitcoin é um Time Vault

Toda carteira Bitcoin é, na essência, um time vault: um segredo criptográfico (chave privada) protegido pela dificuldade computacional de reverter curvas elípticas (ECDSA/secp256k1). Enquanto o hardware atual não consegue derivar a chave privada a partir da chave pública, avanços em computação quântica (algoritmo de Shor) ou clássica poderão eventualmente quebrar essa proteção.

Precedente histórico: as carteiras de Satoshi Nakamoto. Satoshi minerou ~1,1 milhão de BTC entre 2009–2010 usando endereços P2PK (Pay-to-Public-Key) — o formato mais antigo do Bitcoin, onde a chave pública está exposta diretamente no script de saída da transação. Diferente dos formatos modernos (P2PKH, P2WPKH), que publicam apenas um hash da chave pública, os endereços P2PK revelam o blueprint completo para um ataque.

Implicação: ~1,72 milhão de BTC em endereços P2PK antigos (incluindo os de Satoshi) estão vulneráveis a um ataque quântico. Quem construir um computador quântico suficientemente poderoso pode: 1. Colher as chaves públicas expostas do blockchain (são públicas) 2. Executar o algoritmo de Shor para derivar as chaves privadas 3. Transferir os fundos

O protocolo Time Vault torna esse padrão intencional e quantificável: encriptação que é segura hoje mas será quebrável no futuro, com custo previsível via $\mathcal{B}$.

Armazenamento do Vault — GitHub como Banco de Dados

O ciphertext C e os metadados públicos são armazenados em um repositório GitHub público (`time-vault-secrets`), que funciona como um banco de dados flat-file de append-only. Cada vault é salvo como um arquivo JSON nomeado pelo hash de verificação:

$$\texttt{\{V_hex\}.vault.json} = \{C,\ n,\ t_0,\ T\}$$

Campo	Descrição
Nome do arquivo	`{V_hex}.vault.json` — V como identificador único
C	Ciphertext (AES-256-GCM) — o segredo encriptado
n	Número de bits da semente — dificuldade
t_0	Ano de criação
T	Tempo alvo em anos

Leitura (sem autenticação): o repositório é público. Qualquer pessoa pode listar e ler os vaults via GitHub API: - `GET /repos/{owner}/time-vault-secrets/contents/` — lista todos os vaults - `GET /repos/{owner}/time-vault-secrets/contents/{V_hex}.vault.json` — lê um vault específico

Escrita (token público): um Personal Access Token (PAT) fine-grained é hardcoded no cliente com permissão `Contents:write` apenas no repositório `time-vault-secrets`. Qualquer pessoa pode adicionar vaults — isso é intencional. Deleção é impedida por branch protection rules (sem force-push, sem deleção direta). O repositório é um registro público e imutável de vaults.

Relação com a blockchain: a transação Bitcoin de financiamento contém, no OP_RETURN, a URL completa do vault:

```
https://julioflima.github.io/timevault/v/{V_hex}.vault.json
```

Essa URL aponta para a página de decrypt do site, que busca o arquivo `{V_hex}.vault.json` no repositório GitHub. Assim, o blockchain aponta para o arquivo que contém o segredo encriptado — quem encontra S pode seguir a URL, obter C, derivar $K =$ HMAC-SHA256(S, V), e decriptar F.

Fluxo completo: 1. Blockchain (OP_RETURN) → URL com V 2. URL → GitHub repo → arquivo `{V_hex}.vault.json` → C 3. Com S (revelado no witness ou obtido por busca): $K =$ HMAC-SHA256(S, V) 4. $F =$ AES-256-GCM.Dec(K, C) — segredo revelado

---

Mecanismo de Bounty — Hashlock P2WSH

O Time Vault usa uma transação Bitcoin para registrar o vault na blockchain e travar um bounty que só pode ser resgatado revelando S. A mesma transação contém a URL para o arquivo com o ciphertext C (no OP_RETURN), criando um vínculo permanente entre o bounty on-chain e o segredo encriptado off-chain.

Transação de Financiamento (construída pelo Worker):

O usuário paga via QR code ao endereço de doação do projeto. O Worker constrói e transmite a transação de financiamento com as seguintes saídas:

Saída 0 — P2WSH Hashlock (Bounty):

$$\text{Script} = \text{OP_SHA256}\ \langle \text{SHA256}(S) \rangle\ \text{OP_EQUAL}$$

Elemento	Descrição
P2WSH	Pay-to-Witness-Script-Hash — endereço SegWit que referencia um script arbitrário
Hashlock	Trava por hash: os fundos só podem ser gastos fornecendo a pré-imagem do hash
`SHA256(S)`	O hash-alvo — computado a partir de S durante a criação do vault
`OP_SHA256`	Opcode Bitcoin que computa SHA-256 do dado no topo da pilha
`OP_EQUAL`	Verifica se o hash computado é igual ao hash-alvo
Valor	Qualquer quantia acima de 330 sats (dust limit para P2WSH)

Para gastar essa saída, o atacante deve fornecer S no witness data:

$$\text{Witness: } \langle S \rangle$$

O nó Bitcoin executa `SHA256(S)` e verifica contra o hash-alvo no script. Se igual, a transação é válida e os fundos são liberados. S fica permanentemente registrado no blockchain — qualquer pessoa pode ler o witness data da transação de gasto.

Saída 1 — OP_RETURN (Metadados do Vault):

OP_RETURN "`https://julioflima.github.io/timevault/v/{V_hex}.vault.json`"

Elemento	Descrição
OP_RETURN	Marca a saída como não-gastável; usado para armazenar dados arbitrários no blockchain
URL completa	Link clicável em block explorers, aponta diretamente para a página de decrypt do vault
`{V_hex}`	V codificado em hexadecimal (64 caracteres ASCII) — identifica unicamente o vault
n	Não armazenado no OP_RETURN — lido do arquivo vault JSON via URL
`t_0`	Não armazenado — derivado do timestamp do bloco (já registrado pelo Bitcoin)
Custo	0 sats (saída não-gastável, sem dust limit)
Tamanho	~108 bytes (ASCII). Desde Bitcoin Core v30 (Out 2025), OP_RETURN suporta até ~100KB

Custo mínimo total da transação:

Componente	Valor
Bounty (saída P2WSH)	≥ 330 sats (dust limit)
OP_RETURN	0 sats
Fee (~220 vBytes × 1 sat/vB)	~220 sats
Total	~550 sats (~R$0,50 a ~$100K/BTC)

Transação de Gasto — Prova de Quebra On-Chain

Quando alguém quebra o vault (encontra S por busca exaustiva):

$$\text{Witness data: } \langle S \rangle$$

O nó Bitcoin: 1. Pega S do witness 2. Computa `SHA256(S)` 3. Compara com o hash-alvo no script P2WSH 4. Se igual → transação válida → fundos transferidos ao atacante

Verificação pública: qualquer observador pode, a partir de S revelado:

$$V_{\text{check}} = \text{HMAC-SHA256}(S,\ n) \stackrel{?}{=} V$$

Se $V_{\text{check}} = V$ (o V do OP_RETURN na transação de financiamento), está confirmado que o vault foi legitimamente quebrado. O ciclo completo é verificável:

Evento	Dado on-chain	Verificação
Vault criado	OP_RETURN: URL com V	Link para o vault. n do arquivo JSON. t_0 do timestamp do bloco
Bounty travado	Saída P2WSH com hashlock	Fundos travados até S ser encontrado
Vault quebrado	S revelado no witness	HMAC-SHA256$(S,\ n) = V$ ✓

O blockchain se torna um registro de prova-de-quebra: vault existe (V no OP_RETURN) → bounty travado (P2WSH) → vault quebrado (S no witness).

Fluxo de Pagamento — Worker como Intermediário

O usuário paga uma única vez, escaneando um QR code com qualquer carteira Bitcoin. Um Worker (função serverless stateless) constrói a transação de financiamento com todas as saídas necessárias.

Fluxo do usuário (client-side): 1. O site gera um QR code BIP21: `bitcoin:{donation_address}?amount={am` 2. O pagamento do usuário inclui um memo OP_RETURN: `TV-{V_hex}-{P2WSH_address}` 3. Usuário escaneia o QR com qualquer carteira Bitcoin e paga. Pronto — zero complexidade.

Fluxo do Worker (server-side): 1. Monitora o endereço de doação para transações recebidas (via API mempool.space) 2. Parseia o memo OP_RETURN da transação do usuário para extrair `V_hex` e `P2WSH_address` 3. Deduplicação via blockchain (sem banco de dados): consulta a blockchain por qualquer OP_RETURN existente contendo `timevault/v/{V_hex}.vault.json`. Se encontrar → já processado, ignora. 4. Constrói a transação de financiamento com 3 saídas: - Saída 0 — P2WSH hashlock (bounty): 99% do valor recebido (menos fees) para o endereço P2WSH do memo - Saída 1 — OP_RETURN: `https://julioflima.github.io/timevault/v/{V_hex}.vault.json` - Saída 2 — Troco: sats restantes de volta para o endereço do Worker 5. O 1% retido no endereço de doação é a doação ao projeto — nenhuma saída separada necessária 6. Transmite a transação de financiamento via API blockchain

O Worker é stateless: - Sem banco de dados — a blockchain é a fonte de verdade para deduplicação (verificação de existência de OP_RETURN) e saldo (consulta de UTXOs) - O Worker pode crashar e reiniciar sem perder estado - Necessita apenas: uma chave privada (secret do Cloudflare) + endpoint de API blockchain (mempool.space)

Custo para o usuário:

Componente	Valor
Pagamento do usuário ao endereço de doação	valor escolhido pelo usuário
Worker retém 1% como doação	~1% do valor
Worker envia 99% ao bounty P2WSH	~99% menos fees
Fee da tx do Worker (~250 vBytes × 1 sat/vB)	~250 sats
OP_RETURN	0 sats
Pagamento mínimo do usuário	~600 sats (330 bounty + 250 fee + ~20 doação)

Por que Worker? Carteiras comuns via BIP21 QR só suportam envio para um endereço — não conseguem construir transações com múltiplas saídas customizadas (P2WSH + OP_RETURN). O Worker recebe o pagamento simples e o transforma na transação com 3 saídas. O risco custodial é desprezível (valores mínimos de ~600 sats).

Comparação com Carteiras Convencionais

Propriedade	Carteira Bitcoin normal	Time Vault bounty
Segredo	Chave privada (256 bits, fixo)	Semente S (n bits, configurável)
Proteção	ECDSA (curva secp256k1)	HMAC-SHA256 (busca exaustiva)
Chave pública exposta?	P2PK: sim. P2WPKH: só no gasto	Hashlock: `SHA256(S)` no script

Propriedade	Carteira Bitcoin normal	Time Vault bounty
Vulnerável a quântica?	Sim (Shor's algorithm)	Não (hash, não curva elíptica)
Custo de quebra quantificável?	Não (depende de avanço quântico)	Sim: $\$1\text{M} \times 2^{(n-\mathcal{B})}$
Tempo de quebra intencional?	Não (espera-se que nunca quebre)	Sim (degradação natural é o objetivo)
Prova de quebra on-chain?	Gasto da transação	S revelado no witness

(ISSO DEVE PERMANECER INTACTO)

Ano	Hardware	H/s por unidade	Preço	Unidades/$1M	Total H/s	Hashes/ano	Bits	Custo em 2025 p/ igualar
1945	ENIAC	5	$6M	0.16	0.8	2.5×10^{7}	24	$0.000000113687
1950	UNIVAC I	100	$1M	1	100	3.15×10^{9}	31	$0.0000145519
1955	IBM 704	1K	$2M	0.5	500	1.58×10^{10}	33	$0.0000582077
1960	IBM 7090	10K	$3M	0.33	3.3K	1.04×10^{11}	36	$0.000465661
1965	IBM 360/50	50K	$1M	1	50K	1.58×10^{12}	40	$0.00745058
1970	PDP-11	100K	$100K	10	1M	3.15×10^{13}	44	$0.119209
1975	VAX-11/780	500K	$200K	5	2.5M	7.88×10^{13}	46	$0.476837
1980	8086 PC	50K	$5K	200	10M	3.15×10^{14}	48	$1.90735
1985	386	200K	$4K	250	50M	1.58×10^{15}	50	$7.62939
1990	486 DX	5M	$3K	333	1.67G	5.26×10^{16}	55	$244.14
1995	Pentium	50M	$2K	500	25G	7.88×10^{17}	59	$3,906.25
2000	P3 1GHz	500M	$1K	1000	500G	1.58×10^{19}	63	$62,500.00
2005	P4 3GHz [1]	5M	$500	2000	10G	3.15×10^{17}	58	$1,953.13
2008	Core2 Quad	20M	$300	3333	66.7G	2.1×10^{18}	60	$7,812.50
2010	GTX 480 [2]	200M	$500	2000	400G	1.26×10^{19}	63	$62,500.00
2012	GTX 680	400M	$500	2000	800G	2.52×10^{19}	64	$125,000.00
2014	GTX 980	800M	$550	1818	1.45T	4.59×10^{19}	65	$250,000.00

Ano	Hardware	H/s por unidade	Preço	Unidades/$1M	Total H/s	Hashes/ano	Bits	Custo em 2025 p/ igualar
2016	GTX 1080	1.5G	$600	1666	2.5T	7.88×10^{19}	66	$500,000.00
2018	RTX 2080	2.5G	$700	1428	3.57T	1.13×10^{20}	66	$500,000.00
2020	RTX 3080	4.5G	$700	1428	6.42T	2.03×10^{20}	67	$1,000,000.00
2022	RTX 3090 Ti	6G	$1K	1000	6T	1.89×10^{20}	67	$1,000,000.00
2024	RTX 4090	10G	$1.6K	625	6.25T	1.97×10^{20}	67	$1,000,000.00
2025	RTX 5090 [3]	15G	$2K	500	7.5T	2.37×10^{20}	67	$1,000,000.00 [4]

[1] Queda de 63→58: transição de operações genéricas para SHA-256 real — CPUs P4 são ineficientes para hash criptográfico. [2] Início da era GPU: GPUs passam a dominar o custo/hash para HMAC-SHA256. [3] Estimativa baseada em ~1.5× performance da geração anterior, ~1.25× o preço. [4] Por definição, $1M — é o próprio benchmark de referência.

Leituras notáveis:

- O ENIAC de 1945 custou $6M — sua capacidade de hash equivalente hoje custa $0.000000113687.
- O que um IBM 360 fazia com $1M em 1965 (40 bits) custa hoje $0.007 — menos de 1 centavo.
- A queda de 2005 (63→58 bits) marca a transição de benchmarks genéricos → medição real de SHA-256 em CPUs.
- A partir de 2016, o custo/hash estagnou: GPUs mais rápidas, mas proporcionalmente mais caras. O teto de ~67 bits por $1M/ano é estável.
- Cada bit a mais na coluna “Custo em 2025” representa exatamente o dobro — a escala é exponencial.